# Programowanie obiektowe i inżynieria oprogramowania

Lab 01:

- omówienie zakresu zajęć;
- przypomnienie z C++

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

1. Planowo mamy 28 spotkań, łącznie 70*45min, czyli **52,5h**! Postarajmy się wykorzystać ten czas.
2. **Obecność na zajęciach jest obowiązkowa. Dopuszczalna jest <u>jedna</u> nieusprawiedliwiona nieobecność.**
3. Priorytetem jest nauczenie Państwa programowania obiektowego (w C++), UML i GIT.
4. Na zajęciach realizowany będzie projekt algorytmu genetycznego.
5. **Nie ma możliwości odrobienia zajęć w inny dzień, na innej grupie.**
6. **Instrukcje do zajęć będą Państwu udostępnione.**

7. Bardzo ogólny plan laboratoriów:
   - wstęp do programowanie obiektowego w C++ (*kubeczki*),
   - GIT,
   - realizacja projektu algorytmu genetycznego (C++),
   - UML,
   - dalszy rozwój projektu algorytmu genetycznego (C++).

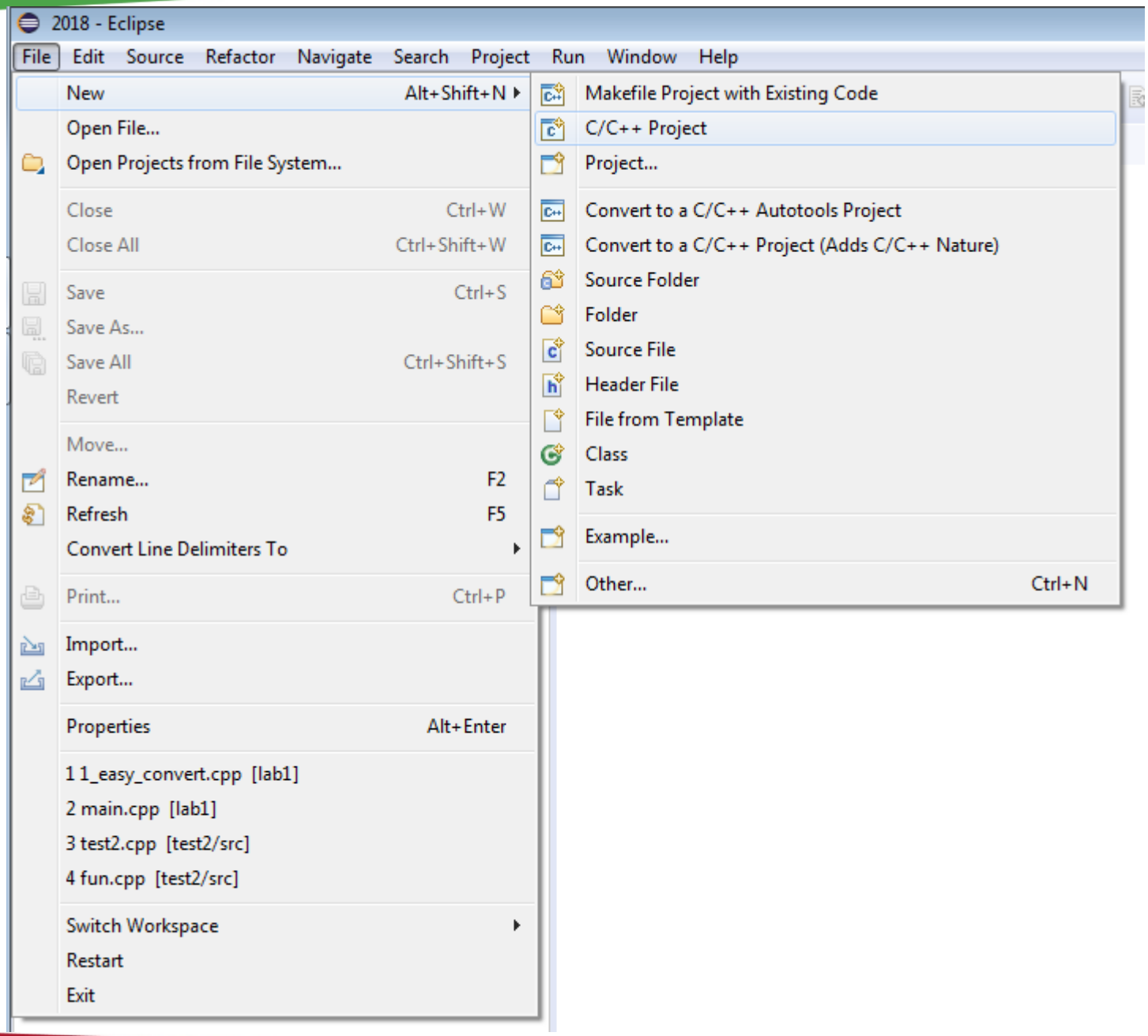
8. Ocena z laboratoriów:
   - **3,0** → wysłanie na zdalne repozytorium mini projektu *kubeczków*,
   - **4,0** → to co na 3,0 plus: wykonanie <u>wszystkich</u> zadań zgodnie z instrukcjami
   - **5,0** → to co na 4,0 plus: wysłanie na zdalne repozytorium skończonego projektu algorytmu genetycznego i projektu z UML.

# Przypomnienie podstaw z C++

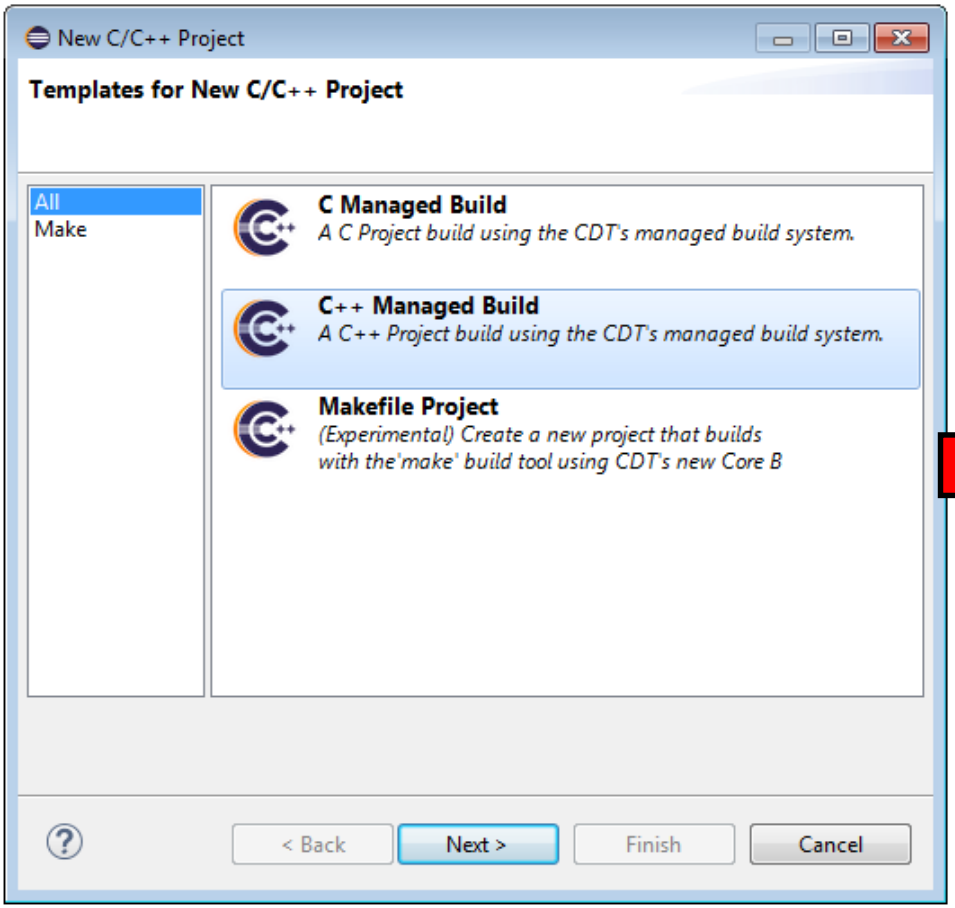
W przypadku problemów z uruchomieniem maszyny wirtualnej na Win10:

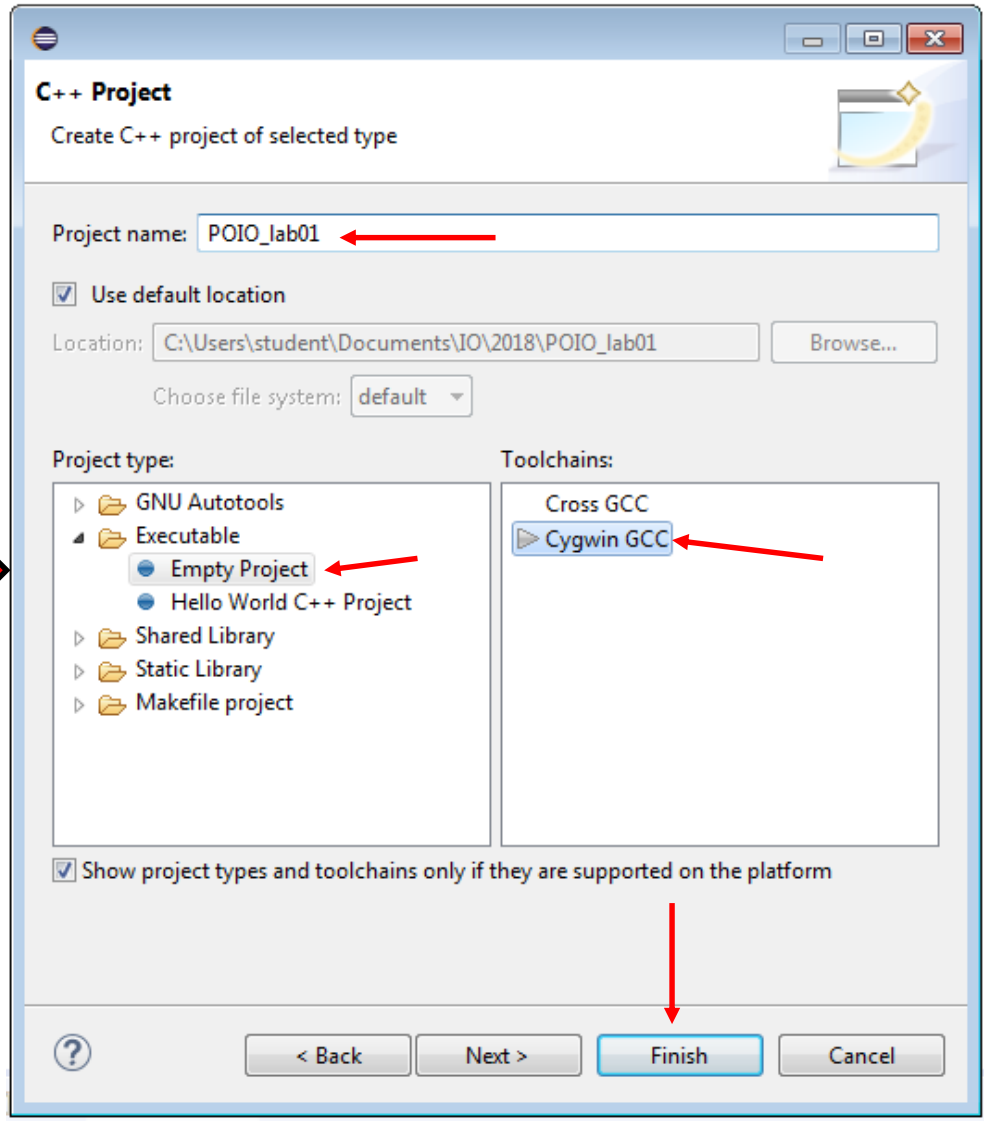
https://www.youtube.com/watch?v=oTMMc9KWtco

# Tworzenie projektu



# Tworzenie projektu



Pierwsza funkcja: konwersja liczby dziesiętnej do postaci binarnej

# Pierwsza funkcja: konwersja liczby dziesiętnej do postaci binarnej

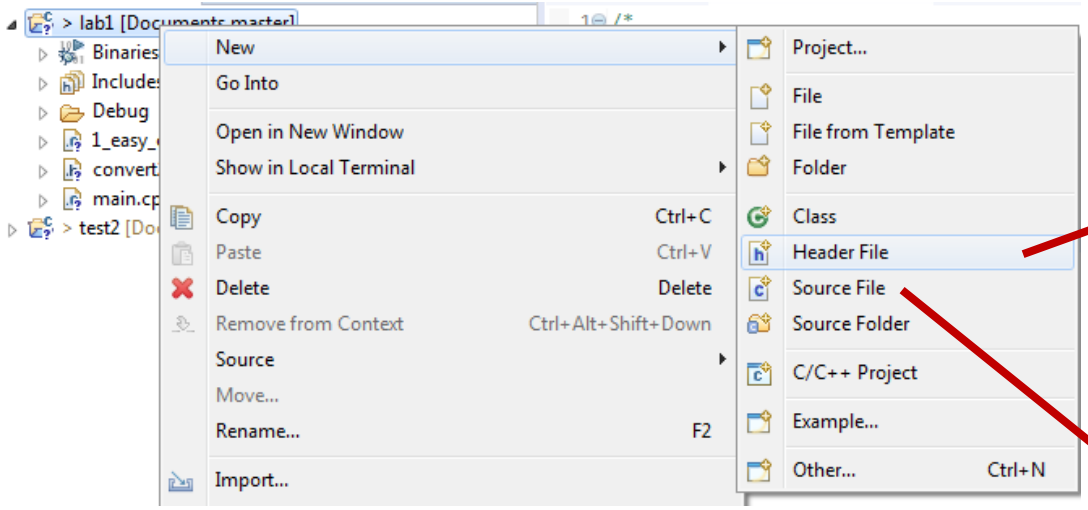
```cpp
#include <iostream>
using namespace std;


void print_binary(int num)
{
          int bin;                  // zmienna do zapisu reszty z dzielenia przez 2

    cout << "result - simple function: ";

    while (num > 0)                 // wykonuj operacje dopóki wartosc argumentu wieksza od 0
    {

        bin = num % 2;              // reszta z dzielenia
        cout << bin;                // wyswietl obliczony fragment liczby binarnej

        num = (num - bin) / 2;      // pozostala liczba do konwersji

    }

    cout << "\n";

}


int main()
{
    cout << "POiO 2021 - lab01 \n\n";

    int num = 18;           // liczba poddana konwersji
    print_binary(num);      // konwersja i wyswietlenie wyniku

    return 0;
}
```

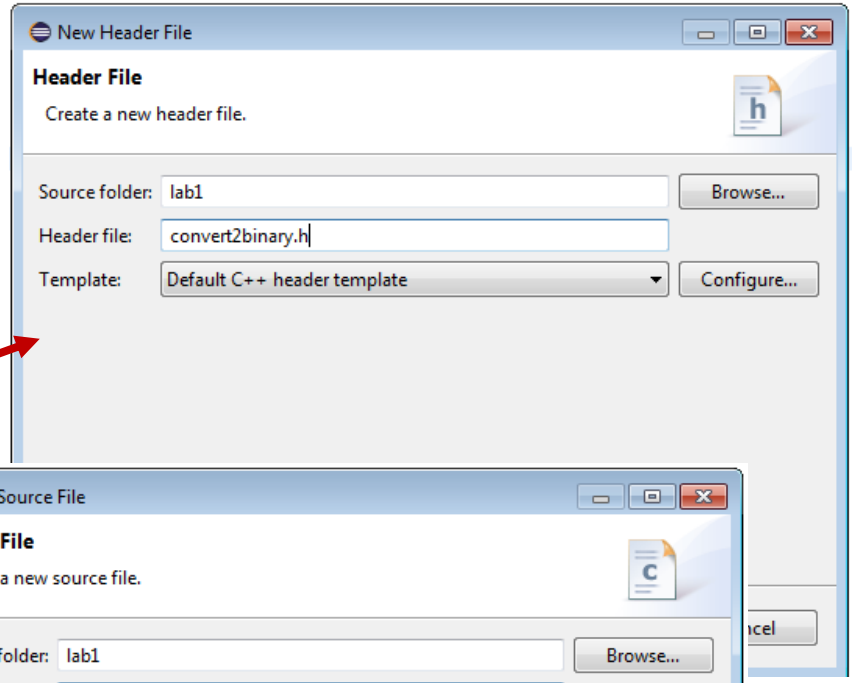
Problems | Tasks | Console | Properties

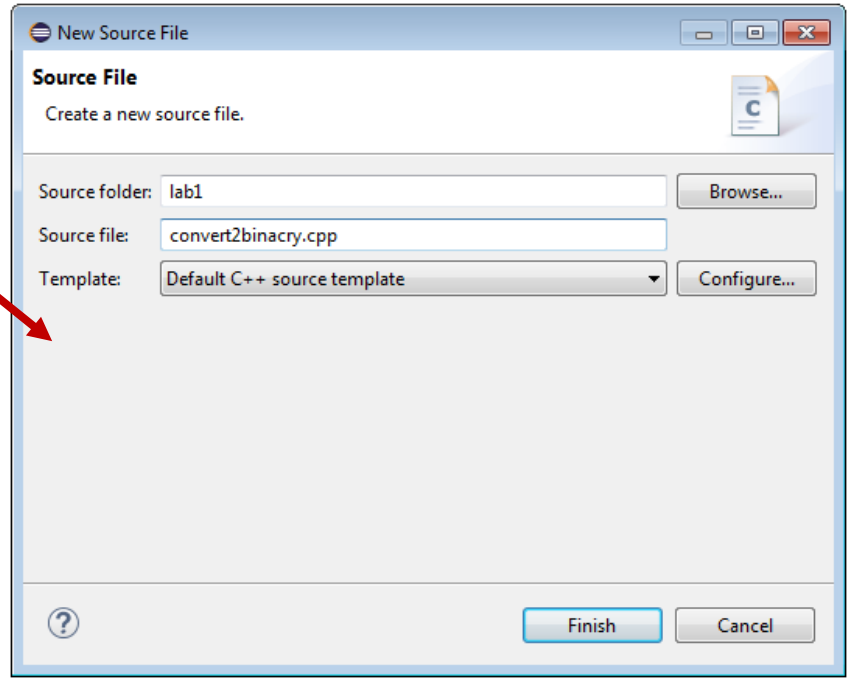
```
<terminated> (exit value: 0) lab1.exe [C/C++ Application] C:\
POiO 2021 - lab01

result - simple function: 01001
```

# Pierwsza funkcja: konwersja liczby dziesiętnej do postaci binarnej

## Funkcje w osobnym pliku



# Pierwsza funkcja: konwersja liczby dziesiętnej do postaci binarnej

## Funkcje w osobnym pliku

Project Explorer

- lab1 [Documents master]
  - Binaries
  - Includes
  - Debug
  - convert2binary.cpp
  - convert2binary.h
  - main.cpp
- test2 [Documents master]

*main.cpp

```cpp
#include <iostream>

using namespace std;

#include "convert2binary.h"


int main()
{
    int num = 18;

    cout << "POiO 2021 - lab01 \n\n";

    print_binary(num);

    return 0;
}
```

*convert2binary.h

```cpp
#ifndef CONVERT2BINARY_H_
#define CONVERT2BINARY_H_


void print_binary(int num);


#endif /* CONVERT2BINARY_H_ */
```

*convert2binary.cpp

```cpp
#include <iostream>
using namespace std;

#include "convert2binary.h"


void print_binary(int num)
{
    short int bin;

    cout << "result - simple function: ";

    while (num > 0)
    {

        bin = num % 2;
        cout << bin;

        num = (num - bin) / 2;

    }

    cout << "\n";

}
```

Problems | Tasks | Console | Properties

```
<terminated> (exit value: 0) lab1.exe [C/C++ Application]
POiO 2021 - lab01

result - simple function: 01001
```

Odwrócona kolejność

# Pierwsza funkcja: tablice

# Pierwsza funkcja: tablice

main.cpp

```cpp
#include <iostream>

using namespace std;

#include "convert2binary.h"


int main()
{
    int num = 4;

    cout << "POiO 2021 - lab01 \n\n";

    print_binary(num);
    print_binary_table(num);

    return 0;
}
```

Dla num = 4

```
POiO 2021 - lab01

result - simple function: 001
result - table: 0100
```

convert2binary.h

```cpp
#ifndef CONVERT2BINARY_H_
#define CONVERT2BINARY_H_


#define SIZE 4

void print_binary(int num);
void print_binary_table(int num);


#endif /* CONVERT2BINARY_H_ */
```

Dla num = 18

```
POiO 2021 - lab01

result - simple function: 01001
result - table: 0010
```

*convert2binary.cpp

```cpp
    {

        bin = num % 2;
        cout << bin;

        num = (num - bin) / 2;

    }

    cout << "\n";

}

// new part
//  ##################################

void print_binary_table(int num)
{
    int bin[SIZE];

    for (int i = SIZE - 1; i >= 0; i--)
    {

        bin[i] = num % 2;
        num = (num - bin[i]) / 2;

    }

    cout << "result - table: ";

    for (int i = 0; i < SIZE; i++)
    {

        cout << bin[i];

    }

    cout << "\n";

}
```

# Pierwsza funkcja: wektory

main.cpp

```cpp
#include <iostream>
using namespace std;

#include "convert2binary.h"


int main()
{
    int num = 18;

    cout << "POiO 2021 - lab01 \n\n";

    print_binary(num);
    print_binary_table(num);
    print_binary_vector(num);

    return 0;
}

```

convert2binary.h

```cpp
#ifndef CONVERT2BINARY_H_
#define CONVERT2BINARY_H_


#define SIZE 4

void print_binary(int num);
void print_binary_table(int num);
void print_binary_vector(int num);


#endif /* CONVERT2BINARY_H_ */

```

convert2binary.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

#include "convert2binary.h"


// new part
// ##################################################

void print_binary_vector(int num)
{

    vector<      int> bin;

    while (num > 0)
    {

        bin.insert(bin.begin(), num % 2);
        num = (num - num % 2) / 2;

    }

    cout << "result - vector: ";

    for (unsigned int i = 0; i < bin.size(); i++)
    {

        cout << bin[i];

    }

    cout << "\n";

}

// ##################################################
```

```
<terminated> (exit value: 0) lab1.exe [C/C++ Application]
POiO 2021 - lab01

result - simple function: 01001
result - table: 0010
result - vector: 10010
```

# Pierwsza funkcje - zadania

Zadania do wykonania:

1. Napisać funkcję przesuwającą bity o określoną liczbę pozycji – uzupełnienie zapisu binarnego zerami z prawej/lewej strony.

2. Zaimplementować funkcję przekształcającą postać binarną do dziesiętnej.